

NTS Pool Protocol and Code Review

Trifecta Tech Foundation has asked Midnight Blue to undertake a review of the proposed NTS pool protocol, which augments the NTS protocol (RFC8915) with the ability to have one Key Exchange Load Balancer (KELB) accept connections from clients, and subsequently securely redirect this client to any of a set of pool member time sources (each NTS/NTP compliant). Backwards compatibility is maintained – existing NTS-compatible clients can transparently use an NTS pool KELB endpoint without need for pool-specific changes. The advantage of pooling is that load balancing of NTS requests becomes possible, and that time sources that are geographically close to the client can be selected, for improved accuracy.

Besides a thorough protocol review, Midnight Blue was asked to closely assess the nts-pool project source code, which serves as an implementation of the draft NTS pool proposal.

The basis for this document is:

- Internet draft: <https://datatracker.ietf.org/doc/draft-ietf-ntp-nts-keyexchange-pool/> version 00
- GitHub: <https://GitHub.com/pendulum-project/nts-pool/>, revision 10066ccfef8e9fbede85650e792bd30b56f1ed0c
- SME interviews

As no comprehensive final report was desired by the client, this document serves as a deliverable; documenting our findings.

Summary

Midnight Blue **found the NTS pool draft specification to be robust** and fit for purpose. The extension is well thought-out, security mechanisms work as intended, and we were not able to identify significant attacks or unintended side-effects that would warrant modification to the draft.

We deem the **code found in the nts-pool GitHub repository to be of high quality**, and while various slight imperfections were found, these were all of limited impact, not inherent to design choices, and easily resolved. The software is well thought-out, and the nature of the Rust programming language further increases the robustness of the software.

As part of the nts-pool code repository, we **find the monitoring service to be a good addition** to keep misconfigured or limited-effort malicious actors from (intentionally or unintentionally) providing clients with incorrect time measurements. However, **some caveats apply**. Since recognition of monitors is almost inherently possible, serious threat actors may accept the arms race of monitoring detection and blacklist avoidance. In this light, we found an attack that allows one pool member to negatively affect scores of other pool members, which might ultimately lead to blacklisting of benign servers (and as a consequence, yield a more dominant position for the attacker).

Threat Model

For the purpose of this assessment, we assume:

- The KELB, management, and monitoring services are run by the pool operators and are considered trusted.
- Clients are considered untrusted.
- Pool members are considered untrusted. That said, we assume the majority of pool members to be benign.

Attacker Model

Time manipulation can be a viable attack vector in select cases. However, it is never a goal in itself, and as such, the complexity of integrating it into a complete attack is high. Examples of attack scenarios include disruption caused by target-internal processes breaking due to time differences, resurrecting the validity of expired key material, or cyber-physical attacks based on phase desynchronization of the power grid.

Powerful adversaries that are able to obtain a forged TLS certificate, thereby breaking the PKI scheme, will be able to attack NTS itself. This puts a clear upper bound on the attacker capabilities that we need to consider. Overall, we consider likely adversaries to have significant technical skill, to target high-value assets, and to use manipulation of NTS as a means to another goal.

Alternatively, less sophisticated adversaries may want to simply cause disruption through less precisely targeted attacks on random clients' NTS requests.

It must be said that clients also bear some responsibility here: they should not only perform multiple measurements on a single time source, but also query multiple independent servers, possibly all from the same NTS pool, in order to achieve high assurance of an accurate time measurement.

Another attacker goal may be the collection of client data, which NTS pool does not aim to protect against.

Security Guarantees

Below are the most essential security guarantees that nts-pool should provide.

- Clients may have high confidence in obtaining a valid time from the pool.
- The choice of a specific time source depends only on the client and the KELB.
- A pool member has no insight into the pool beyond its own secrets and incoming requests.
- Long-term keys (private keys, authentication keys) do not leak beyond their owners.

Attack Scenarios

In this section, we discuss several attack scenarios and their feasibility given the current draft standard and accompanying reference implementation.

Malicious Pool Member Poisons a Targeted Client's Time

- While untargeted modification, for example fooling a set of random clients into accepting an invalid time, may be attractive from a low-effort perspective, the most significant impacts follow from targeted attacks on a client's perception of time.
 - However, a client bears responsibility here: it should perform multiple KELB queries, obtain multiple NTP servers, and compare results to eliminate outliers. In practice, however, this is not always done.
- When random time-source selection is employed by the KELB, an attacker would need multiple colluding time sources to mount an attack, which quickly becomes infeasible as the pool grows larger.
- In practice, however, time sources are selected by the KELB on a geographical basis. An adversary may attempt to introduce multiple malicious time sources in a specific region of interest. This increases the feasibility of ensuring that the target client retrieves a set of predominantly colluding, malicious NTP servers.
- The current geo implementation attaches a great deal of importance to the country. If a country has only one pool member, geo will always return that member, even when it is on the client's `denied_servers` list.
 - This is potentially problematic, as it makes it trivial to attack clients in a country where no, or very few, NTP pool members exist.
 - Legitimate conditions, such as an unfortunate high-jitter route between the client and the NTP pool member, would also yield lower-quality results.
 - We believe better behavior would be to supply the first server using the current strategy, and to supply subsequent servers, for example when `denied_servers` is populated or when keep-alive is used, via a weighted pick from a list of the top 16 matches. This would allow the client's `denied_servers` preference to be respected more strongly.
- This problem is mitigated by the fact that regions, at the continent and country level, are not activated automatically. By default, regions are disabled. Enabling a region is a manual process, to be carried out only when a reasonable number of pool members is present for the region in question. In case of detected tampering, it is also straightforward for the pool operators to disable a region and revert to the global pool.
 - Still, some residual risk remains if a malicious time source, or group of time sources, can affect the reputations of other time sources in the same area.
 - Malicious time sources may also choose to carry out the attack only when a specific client connects. As such, recognizing a target client and only then tampering with the time served through NTP is feasible.
- A vulnerability in the round-robin scheme, an alternative to geo, was discussed, but is not relevant in practice because round-robin time source selection is used only for testing.

There is an inherent trade-off between optimizing for accurate timing measurements and optimizing for security.

- Accuracy: pick servers close to the client, yielding a more predictable set of servers, which is easier for an attacker to target or poison.
- Security: select a less predictable set of servers, which is harder to attack but will, on average, provide lower-quality timing measurements due to the greater distances between client and time source.

Attacker Downgrades the Score of a Competing Pool Member

Under the current *geo* server selection strategy, an attacker can introduce malicious time sources in the client's region in order to increase the likelihood of being selected by the KELB. He may thereby gain the opportunity to ensure a particular time gets adopted by the client, as discussed in the previous section.

Attack probability is also increased if the attacker can negatively impact the reputation of competing pool members. As such, we investigated this venue as well, and found that a feasible approach indeed exists by using the monitor as a vehicle.

We assume that the attacker knows the IP address of the monitor. The monitor's behavior is distinctive enough that it can be distinguished from ordinary clients. For example:

- The first incoming request originates from the monitor, as the time source's reputation is still insufficient for selection by the KELB.
- The monitor reaches out to the pool member both through the KELB and directly.
- The monitor obtains a cookie through NTS, obtains the time and another cookie through NTP, and immediately follows up with another NTP request using the latter cookie.

If we inspect the `probe_ntp` function in the monitor's `probe.rs`, we find that any forged NTP response sent by the attacker is ignored due to cryptographic signature enforcement, so that approach is ineffective.

Furthermore, `probe_ntp` performs the following sequence of invocations:

- `connect(sockfd, ...)`; UDP is connectionless; no packets are sent, but the kernel stores the peer address internally.
- `send(sockfd, ...)`; the NTP request is sent.
- `recv(sockfd, ...)`; the NTP response is received.

In exactly this scenario, if an ICMP Port Unreachable packet is received between the `send()` and `recv()` calls, `recv()` fails with `errno = ECONNREFUSED`, triggering a downgrade in reputation score.

According to RFC 792, an ICMP Port Unreachable packet encapsulates the original IP header plus at least the first 8 bytes of the original datagram. The UDP header fits exactly in these 8 bytes. We found that, on Linux, the ICMP packet is accepted even if the encapsulated IP and UDP headers deviate from the original, as long as both the source and destination IP addresses and ports match. The monitor's IP address and the pool member's IP address and port are known. The only remaining unknown is the monitor's UDP source port, which has 16 bits of entropy and can therefore feasibly be brute-forced.

For validation purposes, a proof of concept for spoofing ICMP Port Unreachable packets was developed and can be provided on request.

Obtaining KELB Secret Key Material

This is a high-impact scenario, as loss of KELB private key material would allow attackers to impersonate the KELB server toward clients, which, through the transitivity of trust, allows them to select any malicious NTP server for the client.

We find private keys exceedingly unlikely to leak, as they are immediately loaded from file into the `rustls` context and do not dwell in code owned by the `nts-pool` codebase. A remote-code-execution-style vulnerability could still reveal those keys, but due to the nature of the codebase, we deem that unlikely.

TLS Misconfiguration

This too is a high-impact scenario, as overly lax validation of TLS parameters could allow an attacker to mount impersonation or man-in-the-middle attacks. We carefully assessed all code related to TLS sockets, namely KELB, management, and monitoring, and confirm that all related code properly enforces the required security properties.

Eavesdropping on NTS Communications

Conceivably, a pool member could want to eavesdrop on communications by other pool members, or on communications between a client and the KELB server. Due to the KELB maintaining connections with pool members through robustly configured mTLS, we identify no risks here. A pool member that does not possess the KELB key material will be unable to breach the integrity or privacy of connections that are not ultimately intended for it.

Findings

This section consists of three main parts. First, we present an analysis of the NTS pool protocol. Then, we present an analysis of the currently implemented monitoring protocol, which is not part of the internet draft. Lastly, we present an analysis of the actual nts-pool implementation in a crate-by-crate fashion. The crate nts-pool-shared is implicitly covered where used by other crates. A summary of the most important issues is provided below, while full details are available in the following subsections.

- The geo server-selection mechanism is predictable and vulnerable to manipulation if insufficient country-level pool members are available.
- Pool member reputation may be negatively affected by an attacker injecting ICMP Port Unreachable packets, exacerbating the previous point.
- nts-pool-ke may give non-RFC8915-compliant responses to a client due to incomplete filtering of pool member KeyExchangeResponse fields.
- If HAproxy is used and the pool KELB endpoint is still directly reachable, a denial-of-service attack through an infinite loop in input processing is possible. An attacker may also be able to forge request source IPs.

NTS Pool Protocol

We mapped out the NTS pool protocol and the monitoring protocol in message-sequence diagrams, and reasoned about the implications of the proposed standard and accompanying implementation. We find the NTS pool protocol to be robust and fit for purpose. While the KELB must be fully trusted, this is acceptable and well documented. The way the KELB interfaces with pool members is well thought out, in line with the original vision of NTS, efficient, and does not provide pool members with more leverage than required. We identified no attacks that apply specifically to the NTS pool protocol additions themselves.

Pool Monitoring Protocol

- The monitor's security goals are not clearly formulated. For example, it is supposed to expose malicious time sources in the pool, while simultaneously allowing malicious time sources to avoid detection because its behavior is recognizable.
- Due to the fundamental nature of the issue, we believe that this goal cannot be achieved in practice to a sufficient degree, and it therefore becomes more of a desired outcome rather than a hard guarantee. Despite the lack of such guarantee, the presence of the monitor can still be beneficial overall. However, care must be taken to ensure that no attacker opportunity exists that runs directly counter to the monitor's goal, as we found to be the case. We believe this should be formulated explicitly as a security consideration.

- Monitoring the behavior of pool members is useful, but it is ultimately an arms race that a monitoring solution cannot truly win. A malicious time source will be able to detect monitoring attempts and/or serve incorrect time to selected clients. Compared to plain NTS, the credibility and reputation of the entire pool is at stake if such attacks are performed. In practice, however, this should be somewhat mitigated by the fact that a client is supposed to perform multiple time measurements. “Global adversary”-style attacks, where a significant part of the pool is colluding, are feasible, but are deemed to require unreasonable effort given the potential gains for the vast majority of client use cases.

Code Review

Crate pool-nts

This crate contains NTS field and message definitions in `record.rs`, as well as functions for building and parsing NTS protocol messages. As such, it was compared closely against the NTS pool draft specification.

We found that the implementation contains some minor incompatibilities with the draft specification. We also found that some behavior permitted by the standard cannot be expressed with the `pool-nts` library. This would be a slight concern if the library were intended as a reference implementation. The development team informed us that this library is not intended as such, and these findings are therefore informational only.

- Good: `NtsRecord::parse()` first reads a 16-bit length field, then parses into a `BufferBorrowingReader` with a maximum `MAX_MESSAGE_SIZE` of 4096 bytes. This is in line with the NTS standard, which requires support for a minimum of 1024 bytes and recommends support for up to 65536 bytes. Overflows are handled gracefully.
- Good: secrets are prevented from leaking through overridden `Debug` implementations. Arguably, `KeyExchangeResponse` cookies could also be classified as secrets.
 - If so, note that trace-level debug output will also include raw cookie values.
- Various structs, such as `ServerInformationRequest`, do not have both `parse()` and `serialize()` methods implemented. This is not a problem, but it would improve consistency, testability, and future reuse of the crate by other parties.
- The NTS pool record type `ListServerNames (0x4006)` exists in the standard but does not appear to be used in the implementation. The standard states: “A server MAY treat this record as unknown for clients that are not authenticated.” However, that implies it should be supported for authenticated KELB <-> pool member communication.
- Message construction and parsing do not fully express all in-spec behaviors:
 - According to the draft, the `AuthenticationToken` for `FixedKeyRequest` or `ServerInformationRequest` is not mandatory, except when keep-alive is also used. The codebase does mandate presence of the `AuthenticationToken` for those requests.
 - The implementation’s behavior remains within spec, but it cannot express everything allowed under the draft standard.
 - It is worth asking why the draft leaves the token optional. Since these requests are meant only for pool-internal purposes, mandatory presence might be more natural.
 - `FixedKeyRequest::parse()` does not enforce that the `AuthenticationToken` precedes the `FixedKeyRequest` record.

Crate nts-pool-ke

This crate implements the key-exchange component. As such, we assessed it for standards compliance and also performed an in-depth review for potential implementation-specific security issues.

- Good: loading of the private keys is done robustly and securely, with minimal risk of unintended leakage.
- Good: validation of pool member certificates is done properly.
- Good: the codebase includes DoS resilience through limits on concurrent connections, default 100, as well as both a client connection timeout, default 5 seconds, and a pool member request timeout, default 2.5 seconds. These mechanisms work.
- However, when HAProxy is used, and a client is able to reach the KELB endpoint (not the regular HAProxy listening service), a bug causes an infinite loop consuming 100% CPU when HAProxy sends a long size field with a shorter and subsequently closes the connection. The `skip_bytes()` function has a while loop that may hang upon encountering EOF early, and will wait for count bytes that will never arrive. CPU usage will spike until the connection times out, which is 5 seconds in the default configuration.
 - This is generally not a problem, as the KELB endpoint should not be reachable when HAProxy is used. As such, the practicality of this as a DoS vector is limited.
 - Additionally, if HAProxy is used and the true KELB endpoint is reachable, there are also source-address-injection issues. In other words, it should be communicated clearly that KELB endpoints must not be publicly reachable when HAProxy is used.
 - Optionally, a simple symmetric secret could be employed to authenticate HAProxy toward the KELB.
- Pool member `KeyExchangeResponse` messages are forwarded by the KELB to the requesting client. Some sanitization and translation is performed to ensure the KELB responds to the client in an RFC8915-compliant manner.
 - The NTS pool `BadRequest` and `UnrecognizedCriticalRecord` errors are filtered out and replaced with an RFC8915-compliant `InternalServerError`. That's good.
 - However; the code still allows non-RFC8915-compliant responses to reach the client. For example, the `keep_alive` field is simply forwarded to the client, as in the following captured KELB -> client message.

```
KeyExchangeResponse {  
  protocol: 0,  
  algorithm: 17,  
  cookies: [[...]],  
  server: Some("localhost"),  
  port: Some(1232),  
  keep_alive: true  
}
```

- Also, as we found during the debriefing, the pool member may send a non-client-supported protocol/algorithm combination, which will equally be forwarded to the client by the KELB.
- User footgun: the config field `upstream-cas` does not define the full CA set; rather, it adds any provided CAs to the existing system CA store. This may be misleading if someone wishes to exclude certain CAs from being accepted, or to set up a system with only a pinned or in-house CA.

- As opposed to the pool-nts crate, no masking of secret values is done. It would be good defense-in-depth either to override Debug on structs carrying secrets, as was done in pool-nts, or to introduce a dedicated secret type such as SecretString with overridden Debug.
- The KELB Docker container restarts every 10 seconds. This is due to the Docker environment, where config-gen regenerates configs every 10 seconds. Watchexec detects the change, and restarts the KELB process. This causes a short (< 0.5 second) window in which the service is unavailable. Potentially, this issue does not exist in the envisioned production environment.

```
ke-1 | 2026-04-22T12:47:09.419836Z INFO nts_pool_ke::pool_ke: Shutting down...
ke-1 | [Running: cargo run -p nts-pool-ke -- --config /app/nts-pool-ke/unsafe.pool.toml]
...
ke-1 | 2026-04-22T12:47:09.775384Z INFO nts_pool_ke::pool_ke: listening on '0k(0.0.0.0:4460)'
```

- Consider this a low-severity informational finding: exploitation of subtle timing side channels was recently shown to be more feasible than previously believed¹. At multiple places in the stack, tokens are compared in a non-constant-time manner, using techniques that largely eliminate network timing randomness. This does not fully relate to NTS pool specifically. However, since the fix is trivial, we recommend replacing these comparisons with constant-time equivalents.
 - Such an approach could be integrated into the aforementioned use of a SecretString-style type. Besides preventing unwanted logging, == operators can be overridden via PartialEq so that only constant-time comparison is used.
 - We considered whether Drop should be overridden to securely wipe secrets from memory. However, this would affect performance, and we deem it unnecessary given the established threat model.

Crate nts-pool-monitor

This crate implements the monitor component and is responsible for establishing the reputation score for each time source. As such, beyond the usual implementation review, we also explored opportunities for a malicious time source to influence that score.

- The monitor exhibits recognizable behavior, and therefore identification by a malicious pool member is straightforward. This is a known issue and cannot be mitigated to a satisfactory extent. As such, the identity of the monitor should be regarded as inherently known.
- User footgun: the config field certificates does not define the full CA set; rather, it adds any provided CAs to the existing system CA store. This may be misleading if someone wishes to exclude certain CAs from being accepted, or to set up a system with only a pinned or in-house CA.
- To be consistent with the pool-nts crate, it would be good defense in depth to override Debug on the authorization key, for example by introducing a SecretString data type. However, it must be noted that a copy of the key is sent to the KELB, and therefore the TLS library may interfere with any guarantee that the key will never end up in debug logs.
- By sending forged ICMP Port Unreachable packets, an attacker has the opportunity to negatively influence the reputation score of a competing pool member. The attack is described in full in the Attack Scenarios section.

Crate nts-pool-management

¹ See for example: <https://portswigger.net/research/listen-to-the-whispers-web-timing-attacks-that-actually-work#local>

We checked the overall structure and conducted a detailed review of route and endpoint authorization checks, database query sanity (for example: whether queries always strongly bind to the authenticated user), and general logic flows.

- All /admin endpoints were confirmed to require a valid admin login session.
- The /management/time-sources/{id} endpoint is a management endpoint but does not require a valid user login. Anyone, logged in or not, could retrieve detailed monitoring information on any time source. It does, however, require a time-source UUID to be known.
- All other /management endpoints required a valid login. Resource ownership also appears to be checked well; a logged-in user cannot access another user's resources.
- Other session-related endpoints also look good, including register, login, logout, forgot-password, and reset-password.
- Monitoring endpoints, get_work and submit, are well authorized. Note that if an attacker could block the monitor's requests to those endpoints, server scores would be frozen. This is not a problem in the currently envisioned use, but it may become one when multiple geographically separated monitors are used. In that case, an adversary may be able to obtain an on-path position and selectively drop such requests.
- As opposed to the pool-nts crate, no masking of secret values is done. It would be good defense in depth either to override Debug on structs carrying secrets, as was done in pool-nts, or to introduce a dedicated secret type such as SecretString with overridden Debug.
- Query structures are good, and the queries implement checks for database entry ownership, for example ensuring that a requested entry belongs to the authenticated entity.
- It is possible to distinguish whether an email address exists. While the reset form provides a non-revealing message, namely "if an account exists, we sent a reset mail," the actual reset-token submission reveals whether the account exists. In such cases, the password is not reset, because the token does not exist, and the user is not notified, because no reset mail was ever requested.
 - <http://localhost:3000/login/reset-password?email=admin@example.com&token=abc> redirects to </login/forgot-password?failed=true>
 - http://localhost:3000/login/reset-password?email=non_existent@example.com&token=abc redirects to /
- UPDATE queries return Ok when executed successfully. However, it is possible that no rows were actually affected. Depending on context, there may be an implicit assumption that the update affects at least one row. As a robustness measure, we recommend revisiting these update queries and returning success only when one or more rows were affected, wherever that expectation exists.
 - Example: for time-source token renewal, a manager may be able to attempt to refresh someone else's token, receive success feedback, but not actually change any token.
- As mentioned before, and unlike in the pool-nts crate, no masking of secret values is done. It could also be good defense in depth to override Display or Debug on structs carrying secrets, or to have a specific datatype such as SecretString.
- Especially in a web application, we would generally recommend constant-time comparisons for secrets, although this would likely be very hard to exploit in practice.

- Password verification is the main exception here, as passwords are hashed using Argon2. In that case, the cost of the hash verification dominates, making any residual timing difference in the final comparison far less relevant.